

Questions et réponses pour préparer le jury

Architecture, React, Flask, MySQL, sécurité, IA locale, RGPD, tests et démarche DevOps.

69 questions adaptées au projet

52 tests backend validés

Projet de Kevin Briquet

Version de préparation : 15 juin 2026

Comment utiliser ce guide

Les réponses sont formulées comme des réponses orales de 30 à 60 secondes. Il ne faut pas les réciter mot pour mot : retiens la logique, cite un exemple du projet et montre que tu connais aussi les limites actuelles.

1. Répondre directement	Commence par une phrase simple qui répond exactement à la question.
2. Donner une preuve	Cite une route, une table, un composant ou un test de HelpMeDraft.
3. Prendre du recul	Termine par une limite ou une amélioration réaliste lorsque c'est utile.

Correspondance avec le titre CDA

Compétence	Éléments de HelpMeDraft à mobiliser
CP1	WAMP, venv, Flask, Vite, React, MySQL, Ollama, .env
CP2	Formulaires, tableau de bord, éditeur, modales, back-office
CP3	Authentification, CRUD, autosauvegarde, versions, IA, quotas
CP4	Découpage fonctionnel, Git, commits et priorisation
CP5	Cahier des charges, acteurs, besoins, parcours et maquettes
CP6	Architecture React / API Flask / MySQL / Ollama
CP7	MCD, clés étrangères, migrations et modèle relationnel
CP8	Requêtes préparées, jointures, transactions et contrôle de propriété
CP9	52 tests Flask, ESLint, build Vite et tests manuels à formaliser
CP10	requirements, package.json, migrations et documentation à compléter
CP11	GitHub et préparation d'une intégration continue

Réponse à éviter : « Tout est sécurisé et totalement conforme. »

Réponse professionnelle : « J'ai mis en place plusieurs protections, je peux les démontrer, et j'ai identifié les contrôles restant à ajouter. »

1. Présenter le projet et le besoin

CP5 - Analyser les besoins et maquetter une application

Q01 Présentez HelpMeDraft en moins d'une minute.

Réponse conseillée. HelpMeDraft est une application web de gestion documentaire destinée à la rédaction de contenus professionnels. Un utilisateur peut créer des dossiers et des documents, rédiger dans un éditeur visuel, sauvegarder automatiquement son travail et utiliser une IA locale pour corriger, reformuler ou compléter son texte. L'application comprend également une authentification sécurisée, un historique des versions et un back-office administrateur.

Q02 À quel besoin métier répond l'application ?

Réponse conseillée. Elle répond au besoin de collaborateurs qui rédigent régulièrement des e-mails, notes ou rapports et souhaitent centraliser leurs documents tout en bénéficiant d'une aide à la rédaction. L'objectif n'est pas de remplacer l'utilisateur, mais de lui faire gagner du temps et de lui laisser la décision finale grâce à une comparaison avant application des suggestions.

Q03 Quels sont les acteurs de l'application ?

Réponse conseillée. Il existe deux rôles. L'utilisateur standard gère uniquement ses propres dossiers, documents, versions et actions IA. L'administrateur accède en plus au back-office : statistiques globales, liste des comptes, rôle, quota IA et activation ou désactivation d'un profil.

Q04 Donnez un parcours utilisateur complet.

Réponse conseillée. L'utilisateur crée son compte, se connecte, crée un dossier puis un document. Il rédige son contenu, applique éventuellement une mise en forme, demande une correction à Ollama, compare les deux versions et choisit ou non d'appliquer la suggestion. Le document est sauvegardé automatiquement et ses anciennes versions restent restaurables.

Q05 Quelles fonctionnalités du cahier des charges sont couvertes ?

Réponse conseillée. Les fonctions principales sont présentes : inscription, connexion, déconnexion, récupération de mot de passe, rôles, éditeur WYSIWYG, trois actions IA, CRUD documentaire, autosauvegarde, historique IA, dossiers, quotas et back-office. Les points encore à formaliser concernent surtout l'audit d'accessibilité, le plan de tests manuel, la CI/CD et la documentation complète d'installation.

2. Environnement et architecture

CP1 et CP6 - Configurer l'environnement et définir l'architecture

Q06 Quelle est la pile technique utilisée ?

Réponse conseillée. Le frontend utilise React avec Vite. Le backend est une API développée avec Python et Flask. Les données sont enregistrées dans MySQL, administré localement avec WAMP et phpMyAdmin. L'intelligence artificielle est fournie localement par Ollama avec un modèle Qwen. Git et GitHub assurent le versionnement.

Q07 Comment l'application est-elle organisée en couches ?

Réponse conseillée. La couche présentation correspond aux composants React. La couche serveur et métier se trouve dans Flask : validations, règles d'accès, quotas et orchestration de l'IA. La couche de persistance repose sur MySQL et les requêtes SQL. Ollama constitue un service externe local appelé par le backend, jamais directement par le navigateur.

Q08 Quel est le rôle de React dans le projet ?

Réponse conseillée. React construit l'interface à partir de composants et gère les états : utilisateur connecté, vue active, formulaires, documents, messages et fenêtres modales. Lorsqu'un état change, React met à jour uniquement les éléments concernés. Cela facilite la séparation des écrans et la réutilisation des comportements.

Q09 Quel est le rôle de Flask ?

Réponse conseillée. Flask expose les routes de l'API. Il reçoit les requêtes JSON du frontend, valide les données, contrôle la session et les autorisations, interroge MySQL, appelle Ollama puis renvoie une réponse JSON avec un code HTTP adapté. Les règles sensibles restent donc côté serveur.

Q10 Pourquoi faut-il deux serveurs pendant le développement ?

Réponse conseillée. Vite sert le frontend React sur le port 5173 et Flask sert l'API sur le port 5000. Ils ont des responsabilités différentes et peuvent évoluer indépendamment. En production, le frontend pourrait être compilé en fichiers statiques et servi derrière le même domaine que l'API.

Q11 **Comment React et Flask communiquent-ils ?**

Réponse conseillée. React utilise fetch pour envoyer des requêtes HTTP à des routes comme /api/login ou /api/documents. Les données sont échangées au format JSON. L'option credentials: include permet au navigateur d'envoyer le cookie de session Flask.

Q12 **Pourquoi avoir choisi une API REST et des réponses JSON ?**

Réponse conseillée. Ce format découple l'interface du backend. Les routes correspondent à des ressources et utilisent les verbes HTTP adaptés : GET pour lire, POST pour créer, PATCH pour modifier et DELETE pour supprimer. JSON est directement exploitable par React et reste facile à tester.

Q13 **À quoi servent le venv et le fichier .env ?**

Réponse conseillée. Le venv isole les bibliothèques Python du projet pour éviter les conflits avec d'autres applications. Le fichier .env contient la configuration variable ou sensible, par exemple les identifiants MySQL, la clé de session, l'adresse d'Ollama et le modèle. Il ne doit pas être envoyé sur GitHub.

3. Interfaces et composants métier

CP2 et CP3 - Développer les interfaces et les composants métier

Q14 Comment avez-vous découpé le frontend React ?

Réponse conseillée. Le composant principal gère l'authentification. Le tableau de bord distribue ensuite l'affichage vers des composants spécialisés : dossiers, documents, éditeur et administration. Chaque composant possède ses états et appelle uniquement les routes nécessaires. Ce découpage limite les responsabilités de chaque fichier.

Q15 Comment fonctionne l'éditeur de texte ?

Réponse conseillée. L'utilisateur voit un éditeur visuel basé sur une zone `contentEditable`. Les boutons appliquent immédiatement le gras, l'italique, le souligné ou un titre. En interne, le contenu est sérialisé en Markdown simple afin de rester lisible dans MySQL, compatible avec l'historique et facile à transmettre au modèle IA.

Q16 Pourquoi conserver du Markdown alors que l'éditeur est visuel ?

Réponse conseillée. Le WYSIWYG améliore l'expérience utilisateur, tandis que le Markdown fournit un format de stockage léger et compréhensible. Cela évite d'enregistrer du HTML arbitraire dans la base. Le frontend reconstruit les éléments visuels à partir du Markdown sans injecter directement une chaîne HTML.

Q17 Comment fonctionne l'enregistrement automatique ?

Réponse conseillée. Après une modification, un délai de 2,5 secondes est lancé. Si l'utilisateur continue à écrire, le délai précédent est annulé. Une comparaison entre l'état actuel et le dernier instantané enregistré évite les requêtes inutiles. Le serveur ne crée une nouvelle version que si le contenu a réellement changé.

Q18 Comment fonctionne l'historique des versions ?

Réponse conseillée. Chaque sauvegarde significative crée une ligne dans `version_document` avec le titre, le contenu, la date, le document et son propriétaire. L'utilisateur peut consulter les versions et en restaurer une. La restauration produit elle-même une nouvelle version afin de ne pas perdre la chronologie.

Q19

Comment sont gérés les dossiers et documents ?

Réponse conseillée. Les opérations suivent un CRUD complet. L'utilisateur crée, lit, modifie et supprime ses ressources. Les documents peuvent appartenir à un dossier ou rester sans dossier. Les listes et les cinq documents récents sont chargés depuis l'API et filtrés selon l'utilisateur connecté.

Q20

Pourquoi afficher une fenêtre de comparaison pour l'IA ?

Réponse conseillée. Une réponse générative peut être incorrecte ou modifier le sens du texte. La fenêtre affiche donc le texte initial et la proposition côte à côte. L'utilisateur conserve le contrôle : il peut fermer la fenêtre, garder son texte ou appliquer la suggestion.

Q21

Que permet le back-office ?

Réponse conseillée. Il affiche les statistiques globales, les comptes inscrits, leurs documents, dossiers, rôles, quotas et statuts. Un administrateur peut modifier le rôle, le quota et activer ou désactiver un compte. Il ne peut pas retirer son propre rôle ni désactiver son propre profil.

4. Base de données et accès SQL

CP7 et CP8 - Concevoir la base et développer les accès aux données

Q22 Présentez votre modèle de données.

Réponse conseillée. La table utilisateur contient l'identité, le mot de passe haché, le rôle, le statut et le quota. Un utilisateur possède plusieurs dossiers et documents. Un document peut appartenir à un dossier, possède plusieurs interactions IA et plusieurs versions.

Q23 Quelles sont les cardinalités principales ?

Réponse conseillée. Un utilisateur peut avoir zéro à plusieurs dossiers et documents. Un dossier appartient obligatoirement à un utilisateur. Un document appartient obligatoirement à un utilisateur et facultativement à un dossier. Une interaction et une version concernent chacune un document et un utilisateur.

Q24 À quoi servent les clés étrangères ?

Réponse conseillée. Elles garantissent l'intégrité référentielle. Par exemple, un document ne peut pas référencer un utilisateur inexistant. Elles rendent également explicites les relations du MCD dans le modèle physique. Pour les versions, une suppression en cascade est prévue.

Q25 Comment protégez-vous les requêtes contre les injections SQL ?

Réponse conseillée. Je n'assemble pas les valeurs de l'utilisateur dans la chaîne SQL. J'utilise les paramètres %s de mysql.connector et je fournis les valeurs séparément. Le connecteur échappe et transmet correctement les paramètres au serveur MySQL.

Q26 Comment empêchez-vous un utilisateur d'accéder au document d'un autre ?

Réponse conseillée. L'identifiant de session est inclus dans les requêtes sensibles. La recherche ne porte pas seulement sur Id_document, mais aussi sur Id_utilisateur. Même en modifiant manuellement l'URL ou la requête HTTP, le document d'un autre compte n'est pas trouvé.

Q27 Pourquoi utiliser des transactions ?

Réponse conseillée. Une opération composée de plusieurs requêtes doit être atomique. Par exemple, la mise à jour d'un document et la création de sa version doivent réussir ensemble. Le commit valide l'ensemble. En cas d'erreur, une version de production devrait effectuer explicitement un rollback.

Q28

À quoi servent les migrations SQL ?

Réponse conseillée. Une migration décrit une évolution reproductible du schéma. Le projet contient par exemple la création de `version_document` et l'ajout du booléen `actif`. Cela évite de dépendre uniquement de manipulations manuelles réalisées dans phpMyAdmin.

Q29

Pourquoi MySQL plutôt qu'une base NoSQL ?

Réponse conseillée. Les données ont des relations fortes et stables : propriétaires, dossiers, documents, versions et interactions. Une base relationnelle apporte les clés étrangères, les jointures et les transactions adaptées à ce besoin. Le NoSQL n'aurait pas apporté de bénéfice suffisant ici.

5. Authentification, sécurité et RGPD

Application sécurisée - Contrôles techniques et limites connues

Q30 Comment les mots de passe sont-ils protégés ?

Réponse conseillée. Ils ne sont jamais stockés en clair. Werkzeug génère un hachage avec un sel, puis la connexion utilise `check_password_hash`. Si la base était volée, le mot de passe original ne pourrait pas être directement lu.

Q31 Quelle est la différence entre hachage et chiffrement ?

Réponse conseillée. Le chiffrement est réversible avec une clé. Le hachage est conçu pour être à sens unique. Pour un mot de passe, on doit seulement vérifier qu'une saisie produit une valeur compatible, pas retrouver le mot de passe initial : le hachage est donc le bon choix.

Q32 Pourquoi avoir choisi les sessions plutôt que JWT ?

Réponse conseillée. Flask conserve l'état d'authentification dans une session signée et le navigateur reçoit un cookie. Pour cette application web classique, cela simplifie la déconnexion et la révocation. JWT serait intéressant pour plusieurs clients indépendants, mais demanderait une stratégie de renouvellement et de révocation.

Q33 Comment le cookie de session est-il sécurisé ?

Réponse conseillée. Il est configuré en `HttpOnly`, donc le JavaScript du navigateur ne peut pas le lire. `SameSite=Lax` réduit certains envois intersites. En production, `Secure` impose HTTPS. La session est limitée à huit heures et signée avec une clé secrète stockée dans l'environnement.

Q34 Comment protégez-vous les routes administrateur ?

Réponse conseillée. Le frontend masque le menu pour un utilisateur standard, mais ce n'est qu'un confort visuel. La vraie protection est le décorateur `admin_required` côté Flask. Il recharge le rôle et le statut depuis la base avant chaque opération sensible et renvoie 403 si l'accès est interdit.

Q35

Pourquoi vérifier le statut actif sur chaque route privée ?

Réponse conseillée. Cela permet une révocation immédiate. Si un administrateur désactive un compte déjà connecté, sa prochaine requête est refusée et sa session est supprimée. Le coût est une requête SQL supplémentaire ; à grande échelle, on pourrait utiliser un cache court ou une stratégie centralisée de révocation.

Q36

À quoi sert CORS dans le projet ?

Réponse conseillée. En développement, React et Flask ont des origines différentes. CORS autorise uniquement le frontend local `http://localhost:5173` à appeler les routes `/api` avec le cookie. Il ne remplace pas l'authentification : les autorisations sont toujours vérifiées par Flask.

Q37

Comment limitez-vous les risques XSS ?

Réponse conseillée. React échappe les chaînes affichées par défaut. L'aperçu et l'éditeur reconstruisent des nœuds contrôlés au lieu d'utiliser `dangerouslySetInnerHTML`. Le collage est ramené en texte simple pour ne pas importer du HTML arbitraire. Une production devrait compléter cela par une politique CSP.

Q38

Comment gérez-vous le risque CSRF ?

Réponse conseillée. Le cookie `SameSite=Lax` apporte une première réduction du risque, mais le projet ne possède pas encore de jeton CSRF explicite. C'est une limite que je sais identifier. Pour renforcer l'application, j'ajouterais un jeton CSRF vérifié sur les requêtes POST, PATCH et DELETE.

Q39

Comment fonctionne la récupération du mot de passe ?

Réponse conseillée. Le serveur génère un jeton signé valable trente minutes. Il contient l'identifiant et une empreinte du hachage actuel. Après modification du mot de passe, l'empreinte change et le lien devient inutilisable. L'envoi SMTP est prévu ; en développement, le lien peut être renvoyé pour les tests.

Q40

Pourquoi la route de mot de passe oublié répond-elle pareil pour une adresse inconnue ?

Réponse conseillée. Cela évite l'énumération des comptes. Si la réponse disait qu'une adresse existe ou non, un attaquant pourrait tester une liste d'e-mails. La route renvoie donc un message générique dans tous les cas.

Q41

Pourquoi désactiver un compte plutôt que le supprimer depuis le back-office ?

Réponse conseillée. La désactivation est une action d'administration réversible : elle bloque l'accès sans détruire les documents. Une suppression RGPD est une procédure différente, définitive et encadrée. L'application distingue donc la suspension opérationnelle du droit à l'effacement.

Q42

Le projet est-il totalement conforme au RGPD ?

Réponse conseillée. Je ne prétendrais pas une conformité complète sans registre de traitement, politique de conservation et analyse juridique. L'IA locale évite l'envoi des documents à OpenAI, ce qui réduit fortement l'exposition. Il reste à formaliser l'information utilisateur, le consentement lié à l'IA, les durées de conservation et la procédure d'effacement.

Q43

Les documents sont-ils chiffrés dans MySQL ?

Réponse conseillée. Non, les mots de passe sont hachés mais le contenu des documents n'est pas chiffré au niveau applicatif. Il faut distinguer ces deux sujets. En production, je prévois le chiffrement du disque ou de la base, des sauvegardes protégées, des comptes MySQL à privilèges limités et éventuellement un chiffrement applicatif pour les contenus sensibles.

6. Intelligence artificielle locale

Composant métier IA - Ollama, prompts, quotas et contrôle utilisateur

Q44 Comment l'IA est-elle intégrée techniquement ?

Réponse conseillée. Le navigateur envoie le texte et l'action à Flask. Flask vérifie le propriétaire du document et son quota, construit les messages puis appelle l'API locale d'Ollama. La suggestion est enregistrée dans l'historique et renvoyée au frontend, qui l'affiche dans une fenêtre de comparaison.

Q45 Pourquoi avoir choisi Ollama plutôt qu'OpenAI ?

Réponse conseillée. Ollama permet d'exécuter le modèle localement, sans clé payante et sans transmettre les documents professionnels à un service tiers. Cela répond mieux à la confidentialité du sujet. Les contreparties sont le poids du modèle et des performances dépendantes de la machine.

Q46 Qu'est-ce que le prompt engineering dans votre application ?

Réponse conseillée. Chaque action possède une instruction système précise : corriger sans changer le sens, reformuler dans un style professionnel ou compléter avec un court paragraphe. Les prompts demandent aussi de conserver le Markdown et de ne retourner que le résultat final. La température faible favorise des réponses stables.

Q47 Comment gérez-vous les hallucinations ou les mauvaises réponses ?

Réponse conseillée. Je ne considère jamais la sortie comme automatiquement correcte. Elle est affichée séparément et doit être acceptée par l'utilisateur. Le prompt limite la tâche, mais un modèle reste probabiliste. Pour aller plus loin, je pourrais ajouter des validations, des tests de prompts et une indication explicite que la proposition doit être relue.

Q48 Que se passe-t-il si Ollama est indisponible ou trop lent ?

Réponse conseillée. L'appel possède un délai maximal configurable. Les erreurs réseau, délais ou réponses invalides sont transformés en réponse 503 et l'interface affiche que l'assistant est indisponible. Le document courant n'est pas remplacé et le quota n'est pas consommé si la génération échoue.

Q49

Pourquoi nettoyez-vous la réponse du modèle ?

Réponse conseillée. Certains modèles Qwen peuvent inclure leur raisonnement interne ou une balise think. Le backend retire cette partie avant affichage. Il rejette aussi une réponse arrêtée par la limite de longueur afin de ne pas proposer un texte incomplet.

Q50

Comment fonctionne le quota IA ?

Réponse conseillée. Chaque compte possède un `quota_restant`. Avant l'appel, Flask vérifie qu'il est positif. Après une génération réussie, une mise à jour conditionnelle le décrémente. L'administrateur peut le modifier. Le cahier des charges évoque un quota journalier ; la version actuelle est un compteur persistant sans remise à zéro automatique.

7. Tests, accessibilité et qualité

CP9 - Préparer et exécuter les plans de tests

Q51 Quels tests automatisés avez-vous mis en place ?

Réponse conseillée. Le backend possède actuellement 52 tests. Ils couvrent l'authentification, les sessions, les mots de passe, les droits d'accès, les dossiers, les documents, les versions, l'IA, les quotas, le back-office et les comptes inactifs. Le frontend est vérifié avec ESLint et une compilation Vite de production.

Q52 Pourquoi simuler la base et Ollama dans les tests ?

Réponse conseillée. Les mocks rendent les tests rapides, reproductibles et indépendants de WAMP ou du modèle local. Ils permettent aussi de vérifier précisément les requêtes et les paramètres transmis. En complément, des tests d'intégration avec une base dédiée seraient utiles pour valider le schéma réel.

Q53 Quelle différence faites-vous entre test unitaire, intégration et test manuel ?

Réponse conseillée. Un test unitaire isole une fonction ou une route avec des dépendances simulées. Un test d'intégration vérifie plusieurs éléments réels ensemble, par exemple Flask et une base de test. Un test manuel reproduit le parcours utilisateur dans le navigateur et permet de contrôler l'affichage, le clavier et les messages.

Q54 Qu'avez-vous prévu pour l'accessibilité ?

Réponse conseillée. Les formulaires utilisent des labels, les boutons ont des noms explicites, les fenêtres possèdent les rôles ARIA adaptés et peuvent être utilisées au clavier. Les contrastes et focus sont travaillés. Je dois encore produire un audit formel Lighthouse ou WAVE et consigner les corrections dans un rapport.

Q55 Quels choix relèvent de l'éco-conception ?

Réponse conseillée. Le frontend évite une grosse bibliothèque d'interface et utilise peu de dépendances. Les données sont chargées par vue et les cinq documents récents sont limités côté SQL. En revanche, un LLM local consomme des ressources : il faut choisir un modèle proportionné, limiter les tokens et garder le modèle chargé seulement lorsque c'est utile.

Q56

Comment gérez-vous les erreurs dans l'interface ?

Réponse conseillée. Les réponses HTTP sont contrôlées et des messages compréhensibles sont affichés. Les boutons passent en état de chargement pour éviter les doubles actions. Une session expirée ou désactivée renvoie vers la connexion. Les actions destructives utilisent une fenêtre de confirmation personnalisée.

8. Gestion du projet, documentation et DevOps

CP4, CP10 et CP11 - Versionner, documenter et automatiser

Q57 Comment avez-vous utilisé Git dans le projet ?

Réponse conseillée. Le projet est versionné depuis son initialisation. Les commits correspondent à des étapes fonctionnelles : backend, connexion MySQL, authentification, inscription, dossiers, documents, IA et historique. Git permet de revenir sur une évolution, comparer les modifications et présenter une progression compréhensible.

Q58 Quels éléments rendent l'installation reproductible ?

Réponse conseillée. Le backend fournit un requirements.txt et le frontend un package.json. Les migrations SQL décrivent les évolutions de la base. Un .env.example et un README global détaillant WAMP, le venv, Ollama et les commandes de lancement restent à finaliser.

Q59 L'application n'est pas déployée : comment justifiez-vous la compétence DevOps ?

Réponse conseillée. La démarche DevOps ne se limite pas à l'hébergement. Elle comprend le versionnement, la configuration séparée, la reproductibilité, les tests automatisés et la préparation d'une intégration continue. Je peux démontrer ces éléments localement et expliquer le pipeline que je mettrais en place.

Q60 Que ferait une GitHub Action pour ce projet ?

Réponse conseillée. À chaque push, elle récupérerait le code, installerait Python et Node, installerait les dépendances, exécuterait les tests Flask, lancerait ESLint et compilerait React. Si une étape échoue, le commit serait signalé. Cela évite d'intégrer silencieusement une régression.

Q61 Pourquoi Docker n'est-il pas encore utilisé ?

Réponse conseillée. Le projet fonctionne aujourd'hui avec WAMP, un venv, Vite et Ollama installés localement. Docker améliorerait la reproductibilité, mais ajouterait du temps et de la complexité. Pour la version titre, je priorise les fonctionnalités et la maîtrise. Je peux toutefois décrire des services frontend, backend et MySQL dans docker-compose.

Q62

Comment prépareriez-vous une mise en production ?

Réponse conseillée. Je compilerais React, utiliserais un serveur WSGI adapté à Windows ou Linux plutôt que le serveur de développement Flask, placerais l'ensemble derrière un reverse proxy HTTPS, sécuriserais les secrets et MySQL, activerais les journaux, les sauvegardes et les en-têtes de sécurité. J'ajouterais aussi les jetons CSRF et les audits.

Q63

Quelles documentations restent nécessaires pour le titre ?

Réponse conseillée. Il faut finaliser le dossier de conception et d'architecture, les diagrammes, le README d'installation, le plan de tests manuel, le rapport d'accessibilité et de sécurité, le journal de veille, la documentation utilisateur et le support de maintenance. Le code et les migrations servent de base technique à ces livrables.

9. Questions pièges et recul technique

Savoir expliquer les limites et proposer une évolution réaliste

Q64 Quelle a été la difficulté principale ?

Réponse conseillée. La difficulté principale a été de faire travailler ensemble plusieurs couches : session Flask, données MySQL, état React et service Ollama. Une modification dans une couche peut avoir des conséquences sur les autres. J'ai progressé en testant chaque parcours progressivement et en ajoutant des tests de non-régression.

Q65 Quel compromis technique avez-vous fait ?

Réponse conseillée. L'éditeur utilise contentEditable et des commandes natives pour rester léger. Cela suffit pour les formats demandés, mais certaines API historiques comme execCommand ne constituent pas le meilleur choix pour un éditeur riche complexe. Une version portfolio pourrait utiliser TipTap, Lexical ou Slate.

Q66 Quelle est la principale faiblesse de sécurité restante ?

Réponse conseillée. L'absence de jeton CSRF explicite est la faiblesse la plus claire. SameSite réduit le risque mais ne remplace pas une protection dédiée. J'ajouterais ensuite une CSP, une limitation de fréquence sur la connexion et la récupération de mot de passe, puis un audit des dépendances.

Q67 Que changeriez-vous si l'application devait accueillir beaucoup d'utilisateurs ?

Réponse conseillée. Je séparerais davantage les modules Flask, ajouterais une couche de services et de dépôt, mettrais en place des index SQL vérifiés, une pagination du back-office, un cache pour certaines lectures et une file de tâches pour les appels IA. Ollama devrait être hébergé sur une machine adaptée ou remplacé par un service scalable.

Q68 Quelles améliorations choisiriez-vous pour une version portfolio ?

Réponse conseillée. Je commencerais par GitHub Actions, une documentation complète et des audits. Ensuite : éditeur riche robuste, recherche documentaire, export PDF ou DOCX, profils et préférences, suppression RGPD, tableau de statistiques plus détaillé, notifications, tests frontend et déploiement démonstratif.

Q69

Pourquoi peut-on dire que le projet couvre le titre CDA ?

Réponse conseillée. Il mobilise l'installation d'un environnement, React, des composants métier, la gestion de projet Git, l'analyse du besoin, une architecture en couches, une base relationnelle, des accès SQL sécurisés et un jeu de tests conséquent. Les compétences de documentation et DevOps sont amorcées mais doivent encore être matérialisées dans les livrables.

Fiche flash avant d'entrer dans la salle

Architecture	React → API Flask → MySQL, avec Ollama appelé uniquement par Flask.
Sécurité	Hash Werkzeug, sessions HttpOnly, requêtes préparées, contrôle de propriété, rôle vérifié côté serveur.
Métier	CRUD documents/dossiers, autosauvegarde, versions, historique IA, quotas et comptes actifs/inactifs.
Qualité	52 tests backend, ESLint et compilation Vite validés.
Limites à assumer	Jeton CSRF, audit RGAA, chiffrement des documents, CI/CD et documentation finale.
Phrase de conclusion	« J'ai privilégié une application fonctionnelle, sécurisée et explicable, puis identifié les améliorations nécessaires pour une production réelle. »

Sources de préparation

Cahier des charges « HelpMeDraft by LexiCorp » - CD2IA 2025/2026.

Code source actuel de HelpMeDraft : React, Flask, MySQL et Ollama.

Référentiel de compétences CDA fourni pour la soutenance.

CNIL : droit à l'effacement et principes de protection des données.

Document de révision - ne remplace pas la démonstration pratique ni les livrables obligatoires du titre.